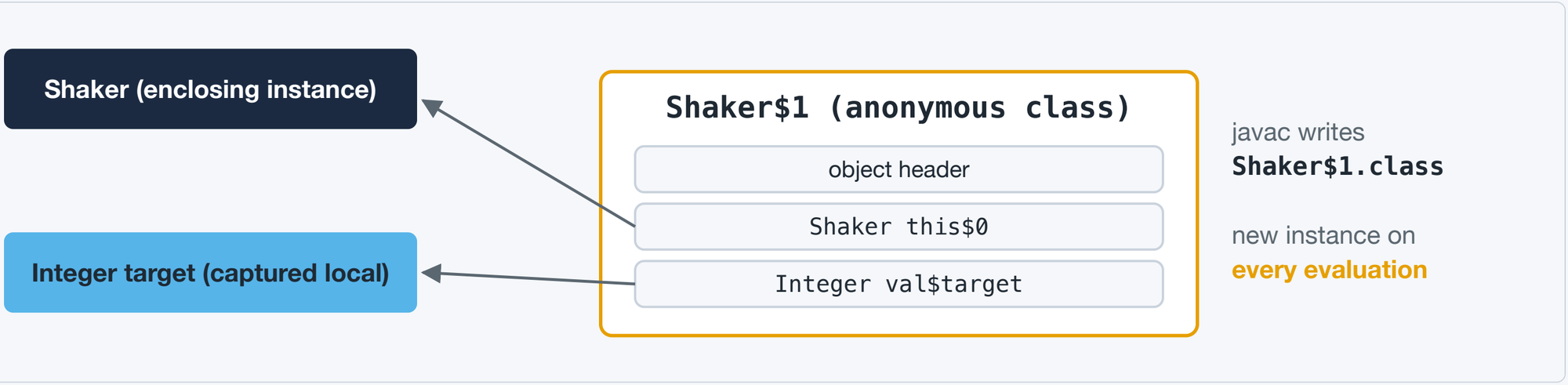
"Dollar" classes

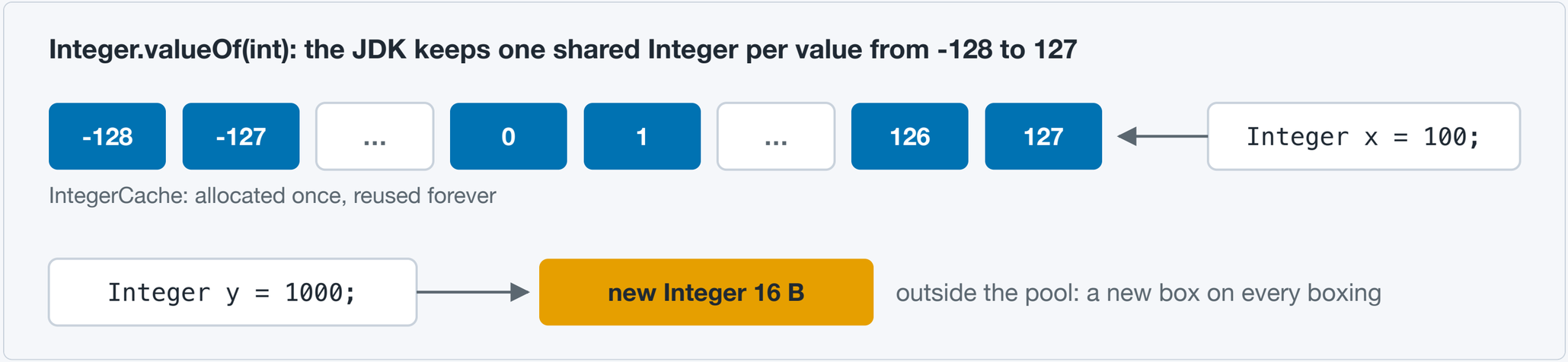
An anonymous class has no name in the code; `javac` names it `Outer$1`.



- A new instance on every evaluation, even when nothing is captured.
- In an instance method it always holds the enclosing `this` (`this$0`).

Object pooling

A pool hands out the same objects again instead of allocating new ones.



- The JDK pools small `Integers`: autoboxing calls `Integer.valueOf(int)`.
- -128 to 127 come from the cache; any other value is a new 16 B object.

Compressed OOPs

A 4-byte offset that HotSpot stores instead of an 8-byte reference, for heaps under 32 GB.

A reference field: uncompressed vs compressed

64-bit address — **8 bytes**

compressed OOP — **4 bytes** — half the size in every field and Object[] slot

```
address = heap base + (offset << 3)
```

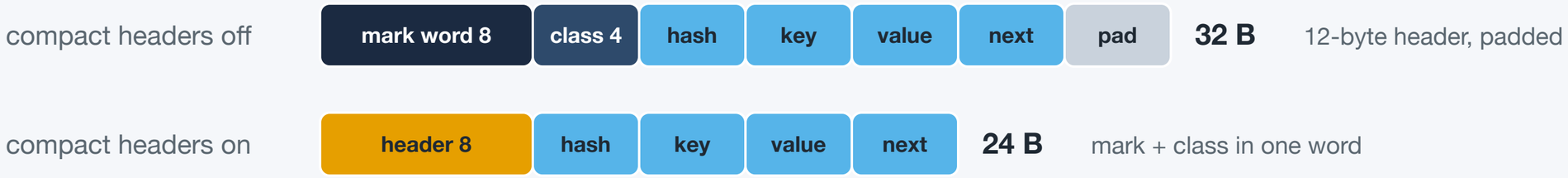
2^32 × 8-byte alignment
= heaps up to 32 GB

- Every reference field and `Object[]` slot shrinks from 8 to 4 bytes.
- On by default (`-XX:+UseCompressedOops`).

Compact object headers

The object header shrinks from 12 bytes to one 8-byte word (JEP 519, JDK 25).

HashMap$Node layout (box width = bytes)

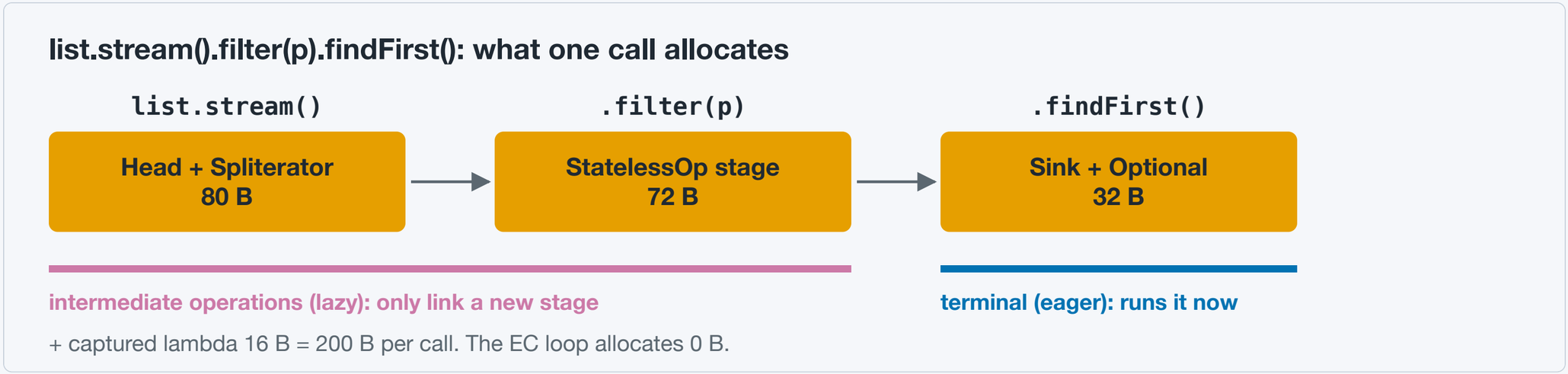


1M HashMap entries: 56.4 MB → 48.4 MB. An Integer stays 16 B either way (12 B rounds up to 16).

- Off by default: `-XX:+UseCompactObjectHeaders`.
- An object only shrinks if it then fits a smaller 8-byte boundary.

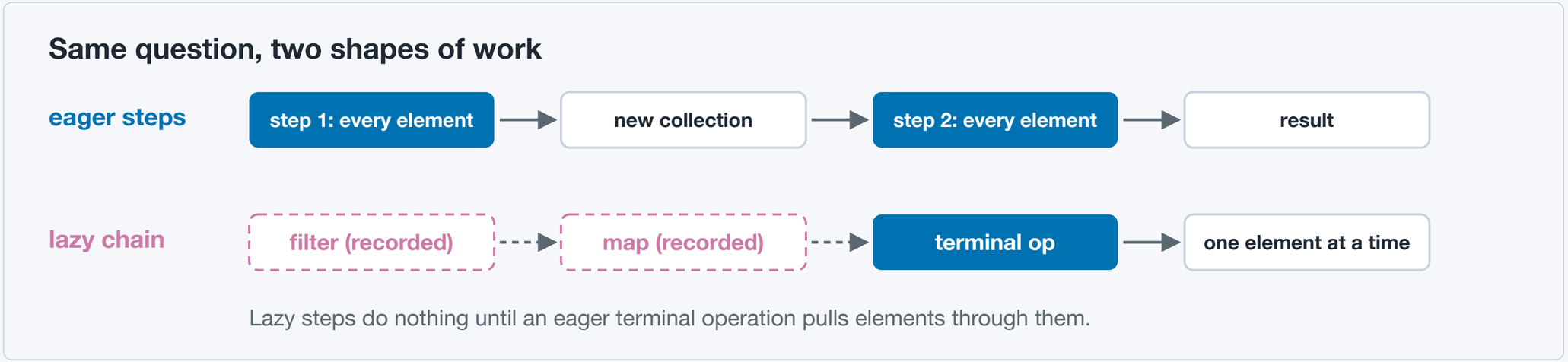
Stream API: what one call allocates

Intermediate operations (`filter`, `map`) are lazy; the terminal operation (`anyMatch`, `count`) is eager and runs the pipeline.



- Every call builds this graph again; escape analysis removes none of it.

Stream API: eager vs lazy



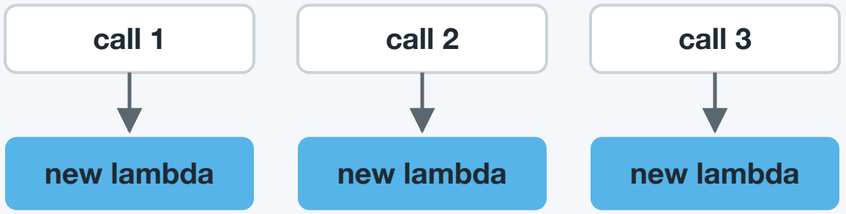
- **Eager**: runs when called and returns a result.
- **Lazy**: recorded; runs one element at a time when a result is needed.

Lambdas: capturing vs fat-free

A capturing lambda holds values from its scope; a fat-free one uses only its parameters.

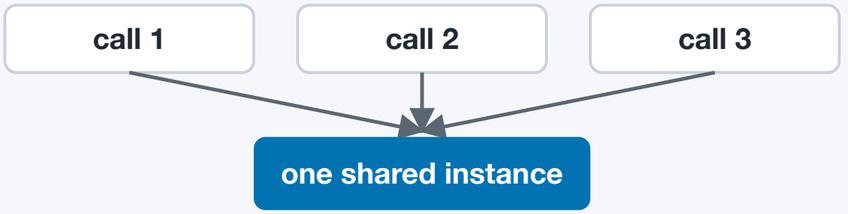
Three calls to the same method

`capturing: each -> each.equals(target)`



3 objects, each holding target

`fat-free: Object::equals, target`



target travels as an argument

- Fat-free: one instance, created once and reused on every call.
- Capturing: a new object that holds the captured values, per evaluation.

Escape analysis

A C2[1] JIT[2] check: when no one outside the method can see a new object, the allocation is removed.

After inlining, C2 asks: can anyone outside this method see the new object?



Escapes = stored in a field, returned, or passed to code that was not inlined. Turn it off: -XX:-DoEscapeAnalysis

- Works only on compiled, inlined code, after warm-up.

---

[1] C2: the HotSpot server compiler, the optimizing compiler for hot methods (C1 is the fast client compiler).

[2] JIT: just-in-time compiler, which turns frequently run bytecode into machine code while the program runs.